TASK 1 REPORT

Data Structures Implementation (C/C++)

Linked List and Stack | Implementation and Testing Report

Student Name	Sohail Khan
Internship	C / C++ Programming Internship
Task Number	1

This report explains the task goal, design, implementation plan, sample test cases, complexity, and steps for building and submitting the project on GitHub.

1. Objective

Implement core data structures from scratch in C++ and demonstrate that each structure supports its main operations. This report focuses on a **singly linked list** and a **stack**, satisfying the task requirement to implement at least two data structures.

- Use clear functions for insertion, deletion, searching, push, pop, and display.
- Demonstrate functionality with small driver/test programs.
- Explain time complexity and provide reproducible build instructions.

2. Data Structure 1 — Singly Linked List

A linked list stores values in nodes. Each node contains data and a pointer to the next node. Unlike an array, its nodes do not need to be stored next to one another in memory.

Operations

- **Insert at front:** create a node, point it to the current head, then update head. Time: O(1).
- **Insert at end:** traverse to the last node and attach the new node. Time: O(n), or O(1) with a maintained tail pointer.
- **Delete by value:** find the node, reconnect the previous node to the next node, and release memory. Time: O(n).
- **Search:** traverse nodes until the value is found or the list ends. Time: O(n).
- **Display:** print each node in order. Time: O(n).

Example

```
Insert 10, 20, 30\nList: 10 -> 20 -> 30 -> NULL\nSearch 20: Found\nDelete 20\nList: 10 -> 30 -> NULL
```

3. Data Structure 2 — Stack

A stack follows LIFO (Last In, First Out): the most recently added item is removed first. An array-backed stack is simple to implement and is useful for learning boundary checks.

Operations

- **Push(x):** add x to the top if the stack is not full. Time: O(1).
- **Pop:** remove and return the top item if the stack is not empty. Time: O(1).
- **Peek:** read the top item without removing it. Time: O(1).
- **Display:** print items from top to bottom. Time: O(n).

Example

```
Push 5\nPush 8\nPush 12\nPop -> 12\nTop -> 8\nStack (top to bottom): 8, 5
```

4. Testing and Sample Results

Run tests for normal operations and edge cases. Expected sample results are shown below; capture your own terminal screenshots after running the implementation.

- Linked list: insert into an empty list, insert several nodes, search for a present and absent value, delete the head, delete a middle node, and delete from an empty list.
- Stack: push until full, attempt one extra push (overflow), pop all items, then attempt pop on an empty stack (underflow).
- Check that deleted nodes are not accessed and dynamically allocated linked-list nodes are released before program exit.

```
Test case                        Expected result\nList insert 10,20,30             10 -> 20 -> 30 -> NULL\nList sea
```

5. Complexity Summary

```
Operation                  Linked List       Array Stack\nInsert at front / Push     O(1)             O(1)\nInsert a
```

The linked list uses O(n) memory for n nodes. The fixed-size array stack uses O(capacity) memory.

6. Build and Run

Save the source file as **task1_data_structures.cpp**. With GCC/G++ installed, run:

```
g++ -std=c++17 -Wall -Wextra -pedantic task1_data_structures.cpp -o task1\n./task1
```

On Windows, run **task1.exe** after compiling. Keep the source code, test inputs, and screenshots in the repository. These commands assume the source file is named as shown.

7. Suggested GitHub Structure and Conclusion

```
task-1-data-structures/\n■■■ task1_data_structures.cpp\n■■■ README.md\n■■■ test-cases.txt\n■■■ screenshots/
```

The project demonstrates how linked lists manage dynamically allocated nodes and how stacks enforce LIFO order. Before submission, verify that the code compiles on your system, replace this report's sample results with actual output if needed, and add the GitHub repository URL to your submission.